

CS2 Analytics Project

1. Set up GitHub repo with Git Bash.
2. Import CS2 data into PostgreSQL.
3. Analyze it with SQL.
4. Optimize a few queries with indexes.
5. Use Python/pandas for deeper analysis.
6. Build a simple win-prediction model.
7. Create a Tableau dashboard.
8. Push updates with Git as you go.
9. Finish with a clean GitHub README.

Flow:

CS2 Data → SQL → Python/ML → Tableau → GitHub

Dataset: https://huggingface.co/datasets/blanchon/opencs2_dataset

Goal: Build a CS2 analytics project using PostgreSQL + SQL + Python/pandas + ML + Tableau + Git/GitHub.

1. Set up project + Git

```
mkdir cs2-analytics  
cd cs2-analytics  
git init
```

Create folders for **data/**, **sql/**, **python/**, **tableau/**, and a **README.md**.

2. Connect to GitHub

```
git add .  
git commit -m "Initial setup"  
git branch -M main  
git remote add origin YOUR_REPO_URL  
git push -u origin main
```

3. Build PostgreSQL database

- Import CS2 dataset
- Create tables/keys
- Clean data

4. SQL analysis

- Opening-kill timing
- Win rates
- Kill differences
- Map/team/player performance
- Use joins, CTEs, window functions, views

5. SQL optimization

- Use **EXPLAIN ANALYZE**
- Add indexes
- Compare query speeds

6. Python/pandas

- Connect to PostgreSQL
- Pull SQL results
- Clean/create features
- Explore relationships

7. Machine learning

- Predict round/match winner
- Compare a few models
- Evaluate accuracy/performance

8. Tableau

- Build an interactive dashboard showing key CS2 findings and model results

9. Push progress regularly

```
git add .  
git commit -m "Add SQL analysis"  
git push
```

10. Finish GitHub README

Include project goal, tools, database diagram, main findings, Tableau screenshots/link, and model results.

Overall flow:

CS2 Data → PostgreSQL → SQL → Python/pandas → ML → Tableau → GitHub

Used pandas to figure out schema before switching to SQL.

Created python/inspect_data.py

Results:

matches

PK: (match_id, map_id)

rounds

PK: (match_id, map_id, round)

FK → matches

kills

PK: kill_id

FK: (match_id, map_id, round) → rounds

round_player

PK: (match_id, map_id, round, player_slot)

FK: (match_id, map_id, round) → rounds

matches 794

rounds 16,527

kills 111,715

round_player 168,294

Built and populated a relational PostgreSQL database from the raw CS2 dataset.

- Installed PostgreSQL and added `psql` to PATH.
- Created a database called `cs2_analytics`.
- Designed 4 tables: `matches`, `rounds`, `kills`, and `round_player`.
- Set primary keys and foreign-key relationships between the tables.
- Loaded the CS2 Parquet data into PostgreSQL using Python.
- Verified the data loaded successfully:
 - 794 match/map rows
 - 16,527 rounds
 - 111,715 kills
 - 168,294 player-round rows
- Saved the schema and loader code in GitHub project and pushed it with Git.

--Does getting the opening kill make a side more likely to win the round?

opening_kill_side	winner_side	rounds
ct	ct	6513
ct	t	2813
t	t	5263
t	ct	1922
(4 rows)		

opening_kill_side	total_rounds	rounds_won	win_rate
t	7185	5263	73.25
ct	9326	6513	69.84
(2 rows)			

Getting the opening kill is strongly associated with winning the round, with T-side opening kills showing a slightly higher round win rate than CT-side opening kills.

-- Does opening-kill timing affect round win rate?

opening_kill_time	rounds	win_rate
0-20 sec	6112	71.34
20-40 sec	5938	71.98
40-60 sec	2423	70.61
60+ sec	2038	70.22
(4 rows)		

The opening-kill timing doesn't change the win rate much.

--kill differential vs. round outcome

kill_difference	rounds	ct_win_rate
-6	7	0.00
-5	490	0.00
-4	1110	0.09
-3	1675	0.12
-2	2349	0.43
-1	1920	3.23
0	482	25.93
1	1265	83.87
2	1981	97.78
3	2242	99.60
4	1946	100.00
5	1057	99.81
6	3	100.00

(13 rows)

When CT finishes a round with fewer kills than T, CT almost always loses. Once CT has even a +1 kill differential, its win rate jumps to about 84%, and at +2 or more it wins almost every round. A side finishing the round with more kills is extremely strongly associated with winning, and even a one-kill advantage matters a lot.

Which maps are more t or ct-sided

map_name	rounds	ct_win_rate	t_win_rate
de_nuke	2606	53.61	46.39
de_overpass	1417	53.00	47.00
de_ancient	2866	52.30	47.70
de_inferno	1877	52.16	47.84
de_mirage	3054	51.57	48.43
de_dust2	3327	48.84	51.16
de_anubis	1380	44.13	55.87

(7 rows)

Map choice affects side advantage, with Nuke leaning CT and Anubis leaning T in this dataset.

How much more often does T win when they get the bomb planted?

has_bomb_plant	rounds	t_win_rate
f	7993	20.31
t	8534	75.80

(2 rows)

T-side round wins are strongly associated with successfully getting the bomb planted.

-- Does opening kill matter once you account for whether T planted bomb or not

opening_kill_side	has_bomb_plant	rounds	t_win_rate
ct	f	5580	8.51
ct	t	3746	62.41
t	f	2413	47.58
t	t	4772	86.23

(4 rows)

Getting the opening kill helps, but getting the bomb planted changes the outcome dramatically even after losing the opening kill.

opening-kill weapon vs round win rate:

opening_weapon	rounds	opening_kill_win_rate
mac10	460	84.57
mp7	138	80.43
hegrenade	175	77.14
galilar	715	76.22
ak47	5111	75.46
m4a1	1057	73.60
awp	2357	73.27
m4a1_silencer	2854	73.16
mp9	556	69.24
glock	600	69.17
elite	220	68.18
famas	230	66.09
usp_silencer	761	59.92
tec9	279	51.97
fiveseven	249	38.55
deagle	364	31.04

(16 rows)

Separate by side:

opening_kill_side	opening_weapon	rounds	win_rate
ct	ak47	1249	77.02
ct	hegrenade	130	75.38
ct	m4a1	1027	72.93
ct	m4a1_silencer	2762	72.92
ct	awp	1620	72.53
ct	mp9	547	68.92
ct	elite	220	68.18
ct	famas	227	65.64
ct	usp_silencer	761	59.92
ct	fiveseven	249	38.55
ct	deagle	184	32.07
t	mac10	449	84.86
t	galilar	670	76.12
t	ak47	3862	74.96
t	awp	737	74.90
t	glock	600	69.17
t	tec9	279	51.97
t	deagle	180	30.00

(18 rows)

So weapon choice is associated with the outcome, but economy/round type is probably influencing this heavily

Does opening-kill advantage change by map

map_name	opening_kill_side	rounds	win_rate
de_ancient	ct	1649	71.07
de_ancient	t	1214	73.06
de_anubis	ct	715	65.45
de_anubis	t	664	78.77
de_dust2	ct	1834	68.16
de_dust2	t	1490	74.83
de_inferno	ct	1090	70.37
de_inferno	t	785	72.99
de_mirage	ct	1727	69.43
de_mirage	t	1322	71.56
de_nuke	ct	1504	71.54
de_nuke	t	1101	70.84
de_overpass	ct	807	72.00
de_overpass	t	609	72.09

(14 rows)

map affects how valuable the opening kill is, especially on Anubis.

Clutch situations vs round wins:

had_clutch_context	rounds	ct_win_rate
f	11074	49.75
t	5453	53.66

(2 rows)

clutch context doesn't change the outcome nearly as much as opening kills or bomb plants

1V1 situations

had_1v1	rounds	ct_win_rate
f	14687	52.07
t	1840	42.83

(2 rows)

T wins 1v1 rounds more often than CT.

headshots vs round wins

headshots	rounds	ct_win_rate
0	457	59.52
1	1635	55.96
2	3177	53.95
3+	11258	49.16

(4 rows)

Misleading

Does having more headshots than the opponent is associated with winning

headshot_difference	rounds	ct_win_rate
-6	1	0.00
-5	103	0.97
-4	524	2.48
-3	1369	4.46
-2	2294	14.86
-1	3005	31.85
0	3070	55.54
1	2619	77.89
2	1925	90.29
3	1065	96.71
4	447	99.55
5	105	99.05

(12 rows)

headshot differential tracks round outcome closely

If the opening kill gets traded within 5 seconds, does the opening-kill advantage mostly disappear?

opening_kill_traded	rounds	opening_side_win_rate
f	11679	78.47
t	4834	54.01

(2 rows)

-Opening kill not traded within 5 sec: opening side wins 78.47%

-Opening kill traded within 5 sec: opening side wins only 54.01%

So a quick trade almost wipes out the opening-kill advantage.

--Does this differ for CT vs T?

opening_kill_side	opening_kill_traded	rounds	win_rate
ct	f	2	0.00
ct	f	6662	77.77
t	t	2664	50.00
t	f	5015	79.44
t	t	2170	58.94
(5 rows)			

A quick trade heavily reduces the value of the opening kill for both sides. It hurts CT even more, dropping the advantage basically to a coin flip.

how long it takes for the trade to happen

trade_time	rounds	opening_side_win_rate
0-2 sec	3224	52.57
2-5 sec	1610	56.89
5-10 sec	1116	55.29
10+ sec	2099	50.36
(4 rows)		

no strong relationship between response time and round outcome once the opponent gets the next kill

Created Optimization.sql

Before:

Seq Scan

12.868 ms

```
----- QUERY PLAN -----
Sort (cost=5401.49..5401.57 rows=31 width=207) (actual time=12.807..12.812 rows=135.00 loops=1)
  Sort Key: round, kill_ordinal
  Sort Method: quicksort  Memory: 59kB
  Buffers: shared hit=3728
-> Seq Scan on kills (cost=0.00..5400.73 rows=31 width=207) (actual time=0.014..12.758 rows=135.00 loops=1)
    Filter: ((match_id = 2391545) AND (map_id = 2))
    Rows Removed by Filter: 111580
    Buffers: shared hit=3725
Planning:
  Buffers: shared hit=217 dirtied=4
Planning Time: 0.580 ms
Execution Time: 12.868 ms
(12 rows)
```

After indexing:

Index Scan

0.085 ms

```
----- QUERY PLAN -----
Index Scan using idx_kills_match_map_round on kills (cost=0.42..63.87 rows=32 width=207) (actual time=0.052..0.064 rows=135.00 loops=1)
  Index Cond: ((match_id = 2391545) AND (map_id = 2))
  Index Searches: 1
  Buffers: shared hit=8 read=3
Planning:
  Buffers: shared hit=229 read=1
Planning Time: 0.267 ms
Execution Time: 0.085 ms
(8 rows)
```

The query became about **151× faster**, or roughly a **99.3% reduction in execution time**.

Given the map, opening-kill side, opening-kill timing, and opening weapon, which side will win the round: CT or T?

Model:

Accuracy: 0.718					
Baseline accuracy: 0.509					
[[1306 375]					
[555 1067]]					
	precision	recall	f1-score	support	
0	0.70	0.78	0.74	1681	
1	0.74	0.66	0.70	1622	
accuracy			0.72	3303	
macro avg	0.72	0.72	0.72	3303	
weighted avg	0.72	0.72	0.72	3303	
Random Forest Accuracy: 0.601					
	precision	recall	f1-score	support	
0	0.61	0.61	0.61	1681	
1	0.59	0.59	0.59	1622	
accuracy			0.60	3303	
macro avg	0.60	0.60	0.60	3303	
weighted avg	0.60	0.60	0.60	3303	

Linear Regression performs better than Random Forest.Feature Importance:

	feature	coefficient
16	cat__opening_weapon_fiveseven	1.187817
36	cat__opening_weapon_tec9	-1.169036
8	cat__opening_kill_side_t	1.009151
7	cat__opening_kill_side_ct	-0.953169
27	cat__opening_weapon_mac10	0.590664
21	cat__opening_weapon_inferno	-0.523177
37	cat__opening_weapon_ump45	-0.501895
28	cat__opening_weapon_mag7	0.441283
18	cat__opening_weapon_glock	-0.415931
12	cat__opening_weapon_cz75a	0.358723
22	cat__opening_weapon_knife_butterfly	0.333900
38	cat__opening_weapon_usp_silencer	0.312998
35	cat__opening_weapon_taser	0.307746
1	cat__map_name_de_anubis	0.297569
20	cat__opening_weapon_hkp2000	0.277079

opening_kill_side_t → strongly pushes prediction toward **T winning**

opening_kill_side_ct → strongly pushes toward **CT winning**

Which side gets the opening kill is one of the strongest predictors of the round winner, while opening weapon and map provide additional context.

With different model, Weapon Class Logistic Regression: (Weapons grouped into classes)

Weapon Class Logistic Regression Accuracy: 0.715

Nearly same accuracy, but better because:

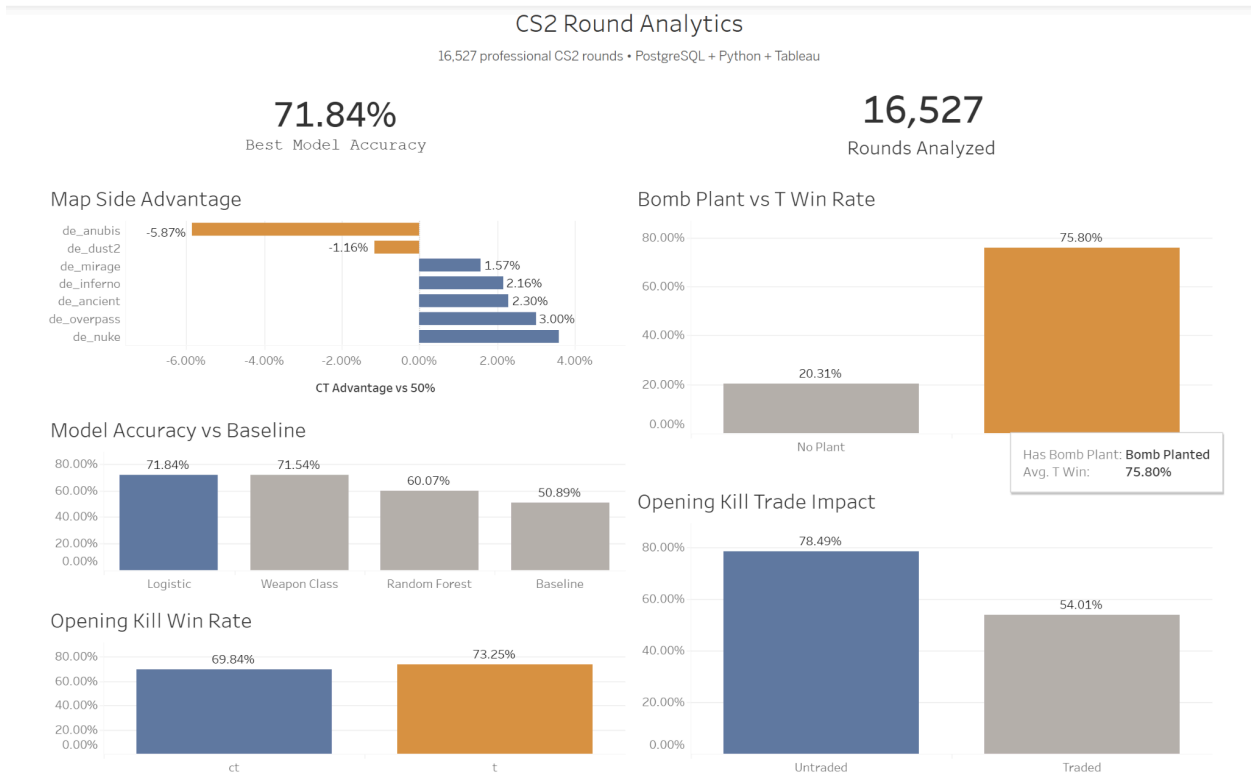
- fewer features after one-hot encoding
- rare guns matter less
- less chance the model overfits to weird small-sample weapons
- feature importance is easier to explain
- you can say things like “SMG opening kills increase/decrease win probability” instead of discussing 15 separate guns

Feature importance:

	Weapon Class Feature Importance	feature	coefficient
8		cat__opening_kill_side_t	0.962685
7		cat__opening_kill_side_ct	-0.880117
9		cat__opening_weapon_class_grenade	-0.365051
10		cat__opening_weapon_class_knife	0.302420
1		cat__map_name_de_anubis	0.284274
17		cat__opening_weapon_class_sniper	-0.281279
12		cat__opening_weapon_class_other	0.235849
15		cat__opening_weapon_class_shotgun	0.235294
14		cat__opening_weapon_class_rifle	-0.211283
18		cat__opening_weapon_class_taser	0.203468
2		cat__map_name_de_dust2	0.132845
6		cat__map_name_de_overpass	-0.132550
16		cat__opening_weapon_class_smg	0.110370
5		cat__map_name_de_nuke	-0.103284
11		cat__opening_weapon_class_machine_gun	-0.086345

Opening kill side matters most; map adds some context; weapon class adds a smaller amount of information.

Tableau Visualization



CS2 Analytics Project — General Overview

This project demonstrates an end-to-end analytics workflow using a professional Counter-Strike 2 dataset. The goal is to analyze factors associated with round outcomes, build predictive models, optimize database performance, and present the results through an interactive dashboard.

The project begins with Parquet data containing match, round, kill-event, and player-round information. The data is loaded into a relational **PostgreSQL** database with primary and foreign key relationships connecting matches, rounds, kills, and player-level statistics.

SQL is used for exploratory and analytical work, including aggregations, joins, CTEs, window functions, and conditional logic. Analyses examine factors such as opening kills, bomb plants, map-side advantages, kill differentials, headshot differentials, clutch situations, and whether opening kills are quickly traded.

One major finding is that the side securing the opening kill wins roughly **70–73% of rounds**. An untraded opening kill corresponds to about a **78.5% win rate**, while a quickly traded opening kill reduces that advantage to around **54%**. Bomb plants are also strongly associated with T-side success.

Database performance is evaluated using PostgreSQL's **EXPLAIN ANALYZE**. Composite indexing reduces the execution time of a representative kill-event query from approximately **12.9 ms to 0.09 ms**, demonstrating practical SQL optimization.

Python and **pandas** are then used to retrieve data directly from PostgreSQL and prepare features for machine learning. Logistic regression and random forest classifiers are compared for predicting whether CT or T wins a round based on information available around the opening kill.

The strongest model, logistic regression, achieves approximately **71.8% accuracy**, compared with a baseline of about **50.9%**. Feature analysis shows that the side securing the opening kill is one of the strongest predictors, while map and weapon information provide additional context.

The final results are presented through an interactive **Tableau** dashboard containing KPI cards, map-side analysis, opening-kill impact, bomb-plant impact, trade analysis, and machine-learning model comparisons.

Technology Stack

PostgreSQL • SQL • Python • pandas • scikit-learn • Tableau • Git • GitHub

Overall Workflow

Raw CS2 Data → PostgreSQL → SQL Analysis & Optimization → Python/pandas → Machine Learning → Tableau Dashboard → GitHub Portfolio Project

CS2 Analytics Project — Detailed Project Overview

This project is an end-to-end data analytics and machine learning project built around the **OpenCS2 professional Counter-Strike 2 dataset**. It demonstrates the full workflow of taking raw data, organizing it in a relational database, analyzing it with SQL, optimizing database performance, processing the results with Python and pandas, building predictive models with scikit-learn, presenting findings in Tableau, and managing the entire project through Git and GitHub.

The central analytical question is:

What factors are associated with winning a professional CS2 round, and how accurately can the round winner be predicted using information available around the opening kill?

The project uses **PostgreSQL, SQL, Python, pandas, scikit-learn, Tableau, Git, and GitHub**.

1. Dataset Selection and Initial Inspection

The project uses the **OpenCS2 dataset** hosted on Hugging Face. Rather than using the full dataset, which includes large amounts of video data, the project focuses on structured Parquet files containing match and gameplay-event information.

Four main datasets are used:

Dataset	Rows	Purpose
Matches	794	Match and map-level information
Rounds	16,527	Round outcomes and aggregate round statistics
Kills	111,715	Individual kill events
Round Player	168,294	Player performance within each round

Before creating the database, Python and pandas are used briefly to inspect the Parquet files. This inspection identifies column names, data types, dataset sizes, and possible relationships between tables.

For example, the inspection reveals that `match_id` alone is not enough to uniquely identify a record in the matches dataset because a single match can contain multiple maps. This leads to the use of a composite identifier involving both `match_id` and `map_id`.

This step is important because database design should follow the actual structure of the data rather than making assumptions about it.

2. PostgreSQL Database Design

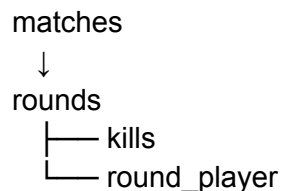
The raw Parquet data is converted into a relational PostgreSQL database named:

`cs2_analytics`

Four tables form the main database:

`matches`
`rounds`
`kills`
`round_player`

The relationship is conceptually:



The `matches` table represents individual maps within professional matches.

Its composite primary key is:

`(match_id, map_id)`

The `rounds` table contains one row for each round played on a map. Its primary key is:

`(match_id, map_id, round)`

The `kills` table contains individual kill events. Each kill has its own unique `kill_id`, while the combination of `match_id`, `map_id`, and `round` connects the event back to the round in which it occurred.

The `round_player` table contains individual player statistics for each round. Its primary key includes:

(match_id, map_id, round, player_slot)

Foreign keys are used to enforce relationships between the tables.

This database structure makes it possible to move from high-level match information down to increasingly detailed information:

match → round → kill event
 → player statistics

This is more representative of a real analytics database than simply loading one CSV into pandas.

3. Data Loading

Because the source files are stored in Parquet format, Python acts as the bridge between the files and PostgreSQL.

A loading script reads each Parquet file with pandas and inserts the records into PostgreSQL using SQLAlchemy and the PostgreSQL driver.

The tables are loaded in dependency order:

matches
↓
rounds
↓
kills
round_player

Loading parent tables before child tables is necessary because foreign key relationships require the referenced records to exist first.

After loading, PostgreSQL contains:

794 match/map records
16,527 round records
111,715 kill events
168,294 player-round records

SQL `COUNT(*)` queries are then used to verify that the database contains the same number of records as the original Parquet files.

At this point, PostgreSQL becomes the primary source for the analysis instead of the raw Parquet data.

4. SQL Exploratory Analysis

The SQL portion examines which gameplay events are associated with winning.

The analysis starts with relatively simple aggregations and gradually introduces more advanced SQL techniques such as:

- GROUP BY
- CASE
- JOIN
- CTE
- window functions
- LEAD()
- aggregate calculations
- conditional aggregation

Opening Kill Advantage

One of the first questions examined is:

How often does the side that gets the opening kill win the round?

The results show:

CT opening kill → ~69.8% round win rate

T opening kill → ~73.3% round win rate

This establishes the opening kill as a strong indicator of round outcome.

5. Opening Kill Timing

The next question examines whether the timing of the opening kill matters.

Opening kills are divided into several time ranges:

0–20 seconds
20–40 seconds
40–60 seconds
60+ seconds

The resulting win rates remain around 70–72% across the different groups.

This produces an interesting finding:

Getting the opening kill matters much more than exactly when the opening kill occurs.

The analysis does not simply search for relationships—it also identifies variables that appear to contribute relatively little additional information.

6. Kill and Headshot Differentials

Round-level kill differential is calculated as:

CT kills – T kills

A very strong relationship appears between final kill differential and the round winner.

For example, when CT finishes the round with even one more kill than T, CT's win rate rises dramatically. Larger positive differentials are associated with nearly guaranteed CT victories.

A similar pattern appears when examining:

CT headshots – T headshots

Positive CT headshot differentials strongly correspond with CT wins, while negative differentials correspond with T wins.

However, these variables are treated carefully.

Because final kill counts and final headshot counts are only known after most or all of the round has already occurred, they are useful for **descriptive analysis** but inappropriate for an early-round predictive model.

Using them as model inputs would create **data leakage**, where the model receives information that would not actually be available at prediction time.

This distinction is important because a model can appear artificially accurate when late-round or post-outcome variables are included.

7. Map Side Advantage

The project also investigates whether individual CS2 maps favor CT or T.

CT win percentage is calculated separately for each map.

The analysis finds examples such as:

Nuke → more CT-leaning

Anubis → more T-leaning

Dust2 → relatively balanced

Rather than displaying only raw CT win percentages, the Tableau dashboard later converts this into a **side advantage metric centered around 50%**.

For example:

CT Advantage = CT Win Rate – 50%

Positive values indicate CT-leaning maps.

Negative values indicate T-leaning maps.

This makes the visualization easier to interpret because the viewer can immediately see how far each map deviates from an even 50/50 split.

8. Bomb Plant Analysis

Another major gameplay variable is whether the T side successfully plants the bomb.

The analysis compares T-side round win rate when a bomb plant occurs versus when no bomb plant occurs.

The results are approximately:

No bomb plant → 20.3% T win rate

Bomb planted → 75.8% T win rate

This is one of the largest differences observed in the analysis.

However, the result is interpreted as an **association**, not proof that planting the bomb alone causes the increase.

Teams that successfully reach a bomb plant are often already in stronger positions than teams eliminated before planting.

The correct interpretation is therefore:

Bomb plants are strongly associated with T-side round success.

9. Opening Kill and Bomb Plant Interaction

The project goes beyond analyzing these variables separately and combines opening kill side with bomb plant status.

For example:

CT gets opening kill + no plant

CT gets opening kill + plant

T gets opening kill + no plant

T gets opening kill + plant

This shows that bomb plants can substantially change the outcome even when T loses the opening duel.

It demonstrates why analyzing combinations of events can be more informative than looking at one variable in isolation.

10. Opening Kill Trades

One of the more advanced SQL analyses investigates what happens when the opening kill is quickly traded.

A **window function** is used to examine the next kill after each kill event.

LEAD() identifies which side gets the next kill.

The query combines:

CTEs

LEAD()

PARTITION BY

ORDER BY

JOIN

CASE

The analysis determines whether the opposing side gets the next kill within five seconds of the opening kill.

The results are approximately:

Opening kill not traded → 78.5% win rate

Opening kill traded → 54.0% win rate

This produces one of the strongest and most meaningful findings in the project:

The opening kill creates a major advantage, but quickly trading that kill eliminates most of the advantage.

The effect is also examined separately for CT and T.

A follow-up analysis groups trade response times into:

0–2 seconds

2–5 seconds

5–10 seconds

10+ seconds

The precise timing does not appear to matter nearly as much as whether the opening advantage gets answered at all.

This is a good example of SQL being used not only for simple reporting but for deriving a more complicated gameplay concept from event-level data.

11. SQL Query Optimization

The project also includes database performance engineering.

A representative query retrieves kill events for a specific match and map:

```
match_id  
map_id  
round  
kill_ordinal
```

Before optimization, PostgreSQL performs a:

Sequential Scan

It checks more than 111,000 rows to retrieve the relevant records.

Using:

EXPLAIN ANALYZE

the measured execution time is approximately:

12.868 ms

A composite index is then created using:

match_id
map_id
round
kill_ordinal

PostgreSQL is analyzed again and begins using an:

Index Scan

The resulting execution time falls to approximately:

0.085 ms

This represents roughly:

99.3% lower execution time

or about a:

151× speed improvement

This section demonstrates that the SQL work includes not only querying data but also understanding how PostgreSQL executes queries and how indexes affect performance.

12. Python and pandas Integration

Once the SQL analysis is established, Python connects directly to the PostgreSQL database.

SQLAlchemy is used to create the database connection, while pandas retrieves query results into DataFrames.

The workflow becomes:

PostgreSQL
↓
SQL query
↓
pandas DataFrame
↓
feature engineering
↓
machine learning

This demonstrates the practical relationship between SQL and pandas.

SQL handles relational storage and database querying.

pandas handles in-memory preparation and modeling workflows.

13. Machine Learning Dataset

The predictive question is:

Given information available around the opening kill, can the model predict whether CT or T will win the round?

The first model uses:

map name
opening kill side
opening kill time
opening weapon

The target variable is converted into binary form:

T win = 1
CT win = 0

Rows with invalid or missing opening-kill information are removed.

The resulting modeling dataset contains approximately:

16,511 rounds

Importantly, late-round variables such as final kill differential are excluded to prevent data leakage.

14. Feature Encoding

Machine learning algorithms require numerical inputs, but several variables are categorical.

Examples include:

```
map_name
opening_kill_side
opening_weapon
```

These are converted using:

OneHotEncoder

A scikit-learn `ColumnTransformer` handles the categorical variables, while numerical variables such as opening kill time are passed through normally.

A scikit-learn `Pipeline` combines preprocessing and modeling so the same transformations are consistently applied during training and prediction.

15. Logistic Regression

The first classification model is **Logistic Regression**.

The dataset is divided into training and test sets using an 80/20 split.

The model achieves approximately:

71.8% accuracy

The baseline accuracy—simply guessing the more common winner—is about:

50.9%

Therefore, the logistic regression model substantially outperforms the baseline.

The classification report also shows reasonably balanced performance between CT and T predictions, although CT wins are identified somewhat more reliably.

The model is not simply guessing one class.

16. Random Forest Comparison

A Random Forest classifier is also trained using the same general inputs.

Its accuracy is approximately:

60.1%

This is better than the baseline but considerably worse than Logistic Regression.

The comparison demonstrates an important machine learning lesson:

A more complex algorithm is not automatically a better model.

Logistic Regression is both more accurate and easier to interpret for this particular dataset.

17. Weapon-Class Model

The original model uses individual weapon names such as:

AK-47
AWP
MAC-10
MP9
Tec-9
Five-SeveN
Deagle
...

This creates many categorical features, including some weapons that appear relatively infrequently.

A second model groups weapons into broader classes such as:

rifle
SMG
pistol
sniper
shotgun
grenade

The weapon-class Logistic Regression model achieves approximately:

71.5% accuracy

This is only slightly below the original model's:

71.8%

The weapon-class version is therefore easier to interpret while sacrificing almost no predictive performance.

18. Model Comparison

The final model comparison is approximately:

Model	Accuracy
Logistic Regression	71.84%
Weapon-Class Logistic Regression	71.54%
Random Forest	60.07%
Baseline	50.89%

The comparison provides evidence that Logistic Regression is the strongest model among those tested.

Rather than presenting one model in isolation, the project demonstrates model selection through comparison.

19. Model Interpretation

The Logistic Regression coefficients are inspected to understand which features influence predictions.

A positive coefficient pushes the prediction toward:

T win

A negative coefficient pushes the prediction toward:

CT win

The strongest feature is the **opening kill side**.

A T opening kill strongly shifts the prediction toward T winning.

A CT opening kill strongly shifts the prediction toward CT winning.

Map and weapon information provide additional predictive context, but their influence is smaller.

This aligns with the SQL analysis and provides consistency between the exploratory analysis and machine learning results.

20. Tableau Data Preparation

The project uses Tableau Public, which does not directly connect to the local PostgreSQL database in the same way as the full Tableau Desktop environment.

Python therefore exports selected PostgreSQL results to CSV files for Tableau.

Examples include:

rounds.csv

opening_trade.csv

model_accuracy.csv

This keeps PostgreSQL as the core analytical database while allowing the results to be visualized publicly.

21. Tableau Dashboard

The final Tableau dashboard is titled:

CS2 Round Analytics

It includes KPI cards and several visualizations.

Key dashboard components include:

Rounds Analyzed

Best Model Accuracy
Map Side Advantage
Bomb Plant vs T Win Rate
Model Accuracy vs Baseline
Opening Kill Win Rate
Opening Kill Trade Impact

The dashboard uses consistent visual logic.

CT-related information uses blue.

T-related information uses orange.

Comparison or secondary states use gray.

The map chart is centered around the 50% win-rate baseline so T-leaning and CT-leaning maps are visually separated.

The model comparison highlights Logistic Regression while keeping weaker models muted.

Technical Tableau-generated labels are removed when unnecessary so the dashboard reads more like a finished analytical product than a collection of raw worksheets.

22. KPI Cards

The dashboard includes high-level KPI cards such as:

16,527
Rounds Analyzed

and:

71.84%
Best Model Accuracy

These provide immediate context before the viewer examines the individual charts.

This is a common dashboard design approach because it communicates scale and the headline result quickly.

23. Tableau Public

The dashboard is published through Tableau Public so it can be opened interactively rather than existing only as a screenshot.

The GitHub README includes both:

dashboard screenshot
interactive Tableau link

This allows a recruiter or reviewer to quickly understand the project from GitHub while still having the option to explore the live visualization.

24. Git and GitHub Workflow

Git is used throughout the project rather than only uploading everything at the end.

The basic development cycle is:

```
edit
↓
git status
↓
git add
↓
git commit
↓
git push
```

Separate commits are created for major stages such as:

database setup
SQL analysis
query optimization
Python analysis
Tableau dashboard
README documentation

This creates a visible project history and demonstrates familiarity with normal version-control practices.

Git Bash is used locally, while GitHub hosts the public repository.

25. Repository Structure

The final repository follows an organized structure similar to:

```
cs2-analytics/  
├── data/  
├── images/  
│   └── dashboard.png  
├── python/  
│   ├── analysis.py  
│   ├── export_tableau.py  
│   ├── inspect_data.py  
│   └── load_data.py  
├── sql/  
│   ├── schema.sql  
│   ├── analysis.sql  
│   └── optimization.sql  
├── tableau/  
│   ├── CS2-Analytics.twb  
│   ├── rounds.csv  
│   ├── opening_trade.csv  
│   └── model_accuracy.csv  
└── README.md
```

Each folder has a distinct purpose:

data → source data
sql → database and analytical queries
python → loading, analysis, and ML
tableau → dashboard and visualization files
images → portfolio screenshots
README → project explanation

This organization makes the project easier for someone else to understand.

26. GitHub README

The README acts as the presentation layer for the repository.

It includes:

- project description
- dashboard screenshot
- interactive Tableau link
- technology stack
- database size
- key findings
- machine learning results
- SQL optimization results
- repository structure
- data source

The goal is to allow someone reviewing the project to understand the major accomplishments without needing to open every SQL and Python file.

The technical files remain available for reviewers who want more detail.

27. Skills Demonstrated

The completed project demonstrates several distinct technical areas within one workflow.

Database engineering: relational schema design, primary keys, foreign keys, PostgreSQL, data loading, and indexing.

SQL analytics: joins, aggregates, conditional logic, CTEs, window functions, and event-level analysis.

SQL performance: `EXPLAIN ANALYZE`, sequential scans, index scans, and composite indexes.

Python: database connections, scripts, data processing, and automation.

pandas: DataFrame manipulation, cleaning, feature preparation, and CSV export.

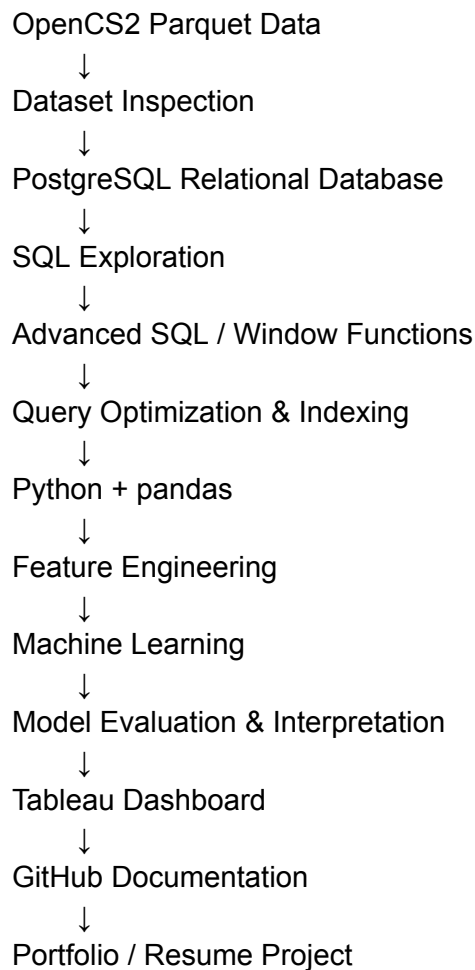
Machine learning: train/test splitting, preprocessing pipelines, one-hot encoding, logistic regression, random forest, classification metrics, baseline comparisons, and feature interpretation.

Data visualization: Tableau calculations, KPI design, comparative visualizations, color consistency, and dashboard design.

Software workflow: Git, GitHub, repository organization, incremental commits, and project documentation.

Overall Pipeline

The complete project can be summarized as:



The result is not simply a SQL project, Python project, machine learning project, or Tableau project. It is an **end-to-end analytics project** demonstrating how the tools fit together in a realistic data workflow.